AutoAgent

Official Technical Specification

A Rust-native local agent runtime for safe, reversible, policy-controlled codebase evolution.

Field	Value
Document Status	Working Draft
Specification Version	0.1.0
Primary Runtime	Rust
Primary Interface	Command Line Interface
Configuration Format	TOML
Execution Model	Local-first, policy-driven, supervised by default
Core Identity	Self-authoring, not uncontrolled self-replicating

1. Executive Summary

AutoAgent is a Rust-native developer agent framework designed to inspect, plan, modify, validate, and evolve software codebases through a local, controlled, reversible runtime. The project centers on a safe mutation engine that can apply structured plans, create snapshots, enforce policies, run validation commands, and preserve a detailed audit trail for every operation.

The long-term signature feature is controlled self-authoring: AutoAgent can work on its own source tree when explicitly permitted. This behavior is implemented as a supervised workflow with policies, run logs, snapshots, patches, validation gates, and rollback rather than as uncontrolled replication or unsupervised propagation.

Official positioning:

> *AutoAgent is a Rust-native local agent runtime for safe, reversible, policy-controlled codebase evolution.*

2. Product Definition

Field	Value
Product Name	AutoAgent
Category	Developer Tooling / Agent Runtime / Codebase Mutation Engine
Primary User	Developers who want a controlled local agent to analyze, modify, validate, and evolve codebases
Primary Use Case	Plan and apply structured source-code changes with reversible patches and validation checks
Flagship Use Case	Controlled self-authoring of the AutoAgent codebase itself

2.1 One-Sentence Description

AutoAgent is a recursive developer agent that can analyze, plan, patch, test, and evolve codebases through a Rust-native, policy-driven local runtime.

2.2 Core Promise

- Analyze the current workspace and detect project structure, language, commands, and relevant files.
- Generate or consume structured implementation plans.
- Validate every file operation against a safety policy before execution.
- Create snapshots and patches before modifying files.
- Run validation commands such as tests, formatting checks, linting, and builds.
- Produce a complete audit trail for every run.
- Support controlled self-authoring only when explicitly enabled.

3. Goals and Non-Goals

3.1 Product Goals

1. Ship as a single reliable Rust CLI binary.
2. Provide deterministic project initialization and workspace metadata creation.
3. Scan and summarize codebases safely while respecting ignore rules.
4. Accept structured JSON plans and apply them through a validated mutation engine.
5. Generate Markdown and JSON run reports.
6. Store snapshots, patches, event logs, and validation results for each run.
7. Support rollback of any applied run.
8. Build toward LLM-assisted planning without making the LLM responsible for direct unsafe mutation.
9. Support future plugin architecture using Rust traits and eventually WASM plugins.

3.2 Non-Goals

- AutoAgent is not an uncontrolled self-replicating process.
- AutoAgent is not designed to persist secretly, spread across machines, or modify repositories outside approved workspace boundaries.
- AutoAgent must not execute arbitrary commands without policy validation and explicit approval when required.
- AutoAgent must not modify environment files, SSH material, Git internals, or system paths by default.
- AutoAgent must not push changes to remote repositories or deploy production changes without a future explicit workflow designed for that purpose.

4. Rust-Native Architecture

AutoAgent should be implemented as a Rust workspace containing separate crates for CLI behavior, core runtime behavior, and future plugin SDK behavior. This keeps the user-facing interface independent from the mutation engine and allows the engine to be tested directly.

```
autoagent/
  Cargo.toml
  README.md
  LICENSE
  .gitignore

  crates/
    autoagent-cli/
      Cargo.toml
      src/
        main.rs
        commands/
          mod.rs
          init.rs
          analyze.rs
          plan.rs
```

```
      apply.rs
      run.rs
      evolve.rs
      patch.rs
      revert.rs
      memory.rs
      doctor.rs
      config.rs

autoagent-core/
  Cargo.toml
  src/
    lib.rs
    runtime/
      mod.rs
      agent_runtime.rs
      agent_loop.rs
      task_context.rs
      run_state.rs
    config/
      mod.rs
      config_loader.rs
      config_schema.rs
      default_config.rs
    analysis/
      mod.rs
      project_analyzer.rs
      file_scanner.rs
      dependency_analyzer.rs
      source_map_builder.rs
    planning/
      mod.rs
      planner.rs
      plan.rs
      plan_reader.rs
      plan_writer.rs
      plan_validator.rs
    editing/
      mod.rs
      file_editor.rs
      file_operation.rs
      diff_builder.rs
      patch_writer.rs
      snapshot_manager.rs
    validation/
      mod.rs
      command_runner.rs
      validation_report.rs
    safety/
      mod.rs
      policy_engine.rs
      path_guard.rs
      command_guard.rs
      approval_gate.rs
    memory/
      mod.rs
      memory_store.rs
      project_memory.rs
      decision_log.rs
    logging/
```

```
      mod.rs
      logger.rs
      run_logger.rs
      event_log.rs
    git/
      mod.rs
      git_client.rs
      branch_manager.rs
    error/
      mod.rs
      autoagent_error.rs

  autoagent-plugin-sdk/
    Cargo.toml
    src/
      lib.rs
      plugin.rs
      tool.rs
      schema.rs

.agent/
  memory/
  plans/
  runs/
  patches/
  logs/
  reports/
  tools/
```

4.1 Crate Responsibilities

Crate	Responsibility
autoagent-cli	Parse CLI arguments, display output, ask confirmations, format reports, and call autoagent-core.
autoagent-core	Load config, analyze projects, validate policies, snapshot files, apply operations, run commands, write logs, manage memory, create patches, and revert runs.
autoagent-plugin-sdk	Define future plugin contracts, tool schemas, plugin manifests, and eventual WASM-compatible extension contracts.

5. Command-Line Interface

```
autoagent init
autoagent doctor
autoagent analyze
autoagent plan "add plugin support"
autoagent apply .agent/plans/add-plugin-support.plan.json
autoagent run "fix failing tests"
autoagent evolve "improve the planner"
autoagent patch list
autoagent patch show <run-id>
autoagent revert <run-id>
autoagent memory show
autoagent config show
```

Command	Purpose	Default Write Behavior
init	Create Autoagent.toml and .agent	Writes initialization files after

	workspace folders.	confirmation or --yes.
doctor	Check Rust, Cargo, Git, config, permissions, commands, and workspace health.	Read-only.
analyze	Scan the project and write a project analysis report.	Writes report files only.
plan	Create or import a structured plan.	Writes plan files only.
apply	Apply a structured plan through snapshots, policy validation, and validation commands.	Writes only approved planned changes.
run	Plan, apply, validate, and optionally repair in one workflow.	Supervised by default.
evolve	Operate on AutoAgent source itself when self-modification is enabled.	Plan-only by default.
patch	List or display stored patches.	Read-only unless saving a new patch.
revert	Restore files from run snapshots or reverse patches.	Writes rollback changes.
memory	Show, rebuild, add, or remove project memory entries.	Depends on subcommand.

6. Configuration Specification

The official configuration file is Autoagent.toml. TOML is preferred because it is easy to read, stable, Rust-native, and safe compared with executable JavaScript configuration files.

```
[project]
name = "autoagent"
type = "rust-cli"
language = "rust"
package_manager = "cargo"

[agent]
mode = "supervised"
allow_self_modification = false
max_steps_per_run = 8
require_approval_before_write = true
require_approval_before_command = true

[workspace]
root = "."
include = [
  "crates/**/*.rs",
  "src/**/*.rs",
  "tests/**/*.rs",
  "Cargo.toml",
  "README.md",
  "Autoagent.toml"
]
exclude = [
  "target/**",
  ".git/**",
  ".agent/runs/**",
  ".agent/patches/**",
  ".agent/logs/**",
  ".env",
```

```
  ".env.*"
]

[commands]
test = "cargo test"
lint = "cargo clippy --all-targets --all-features -- -D warnings"
format = "cargo fmt --all -- --check"
build = "cargo build"

[safety]
allowed_write_paths = [
  "crates/",
  "src/",
  "tests/",
  "README.md",
  "Cargo.toml",
  "Autoagent.toml"
]
blocked_write_paths = [
  ".git/",
  "target/",
  ".env",
  ".env.local",
  ".ssh/",
  "/",
  "../"
]
allowed_commands = [
  "cargo test",
  "cargo build",
  "cargo fmt --all -- --check",
  "cargo clippy --all-targets --all-features -- -D warnings",
  "git status",
  "git diff",
  "git branch",
  "git checkout"
]
blocked_commands = [
  "sudo",
  "rm -rf /",
  "curl",
  "wget",
  "ssh",
  "scp",
  "chmod 777",
  "chown"
]

[memory]
enabled = true
directory = ".agent/memory"

[logging]
directory = ".agent/logs"
level = "info"

[patches]
directory = ".agent/patches"
create_before_write = true
```

```
[runs]
directory = ".agent/runs"
```

7. Agent Runtime Loop

10. Receive objective or plan file.
11. Load Autoagent.toml.
12. Validate workspace root and policy boundaries.
13. Analyze project files and project metadata.
14. Load project memory.
15. Create a task context and run identifier.
16. Generate or read a structured plan.
17. Validate all plan operations against path and command policies.
18. Ask for approval when supervised policy requires approval.
19. Create snapshots of touched files before writing.
20. Apply file operations through the file editor.
21. Create patch and event logs.
22. Run validation commands.
23. Create validation report.
24. Write final summary and update memory if appropriate.

8. Core Data Structures

8.1 Run State

```
// crates/autoagent-core/src/runtime/run_state.rs

use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Eq)]
pub enum RunState {
    Created,
    LoadingConfig,
    AnalyzingProject,
    LoadingMemory,
    Planning,
    AwaitingApproval,
    Snapshotting,
    ApplyingChanges,
    Validating,
    Repairing,
    Completed,
    Failed,
    Reverted,
}
```

8.2 File Operation

```
// crates/autoagent-core/src/editing/file_operation.rs
```

```
use camino::Utf8PathBuf;
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Eq)]
pub enum FileOperationKind {
    Create,
    Write,
    Replace,
    Append,
    Delete,
    Rename,
    CreateDirectory,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct FileOperation {
    pub kind: FileOperationKind,
    pub path: Utf8PathBuf,
    pub destination_path: Option<Utf8PathBuf>,
    pub reason: String,
    pub before_hash: Option<String>,
    pub after_hash: Option<String>,
    pub content: Option<String>,
}
```

8.3 Task Context

```
// crates/autoagent-core/src/runtime/task_context.rs

use crate::config::config_schema::AutoAgentConfig;
use crate::runtime::run_state::RunState;
use camino::Utf8PathBuf;
use chrono::{DateTime, Utc};
use serde::{Deserialize, Serialize};
use uuid::Uuid;

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct TaskContext {
    pub id: Uuid,
    pub run_id: String,
    pub objective: String,
    pub root_directory: Utf8PathBuf,
    pub mode: AgentMode,
    pub self_modification: bool,
    pub state: RunState,
    pub config: AutoAgentConfig,
    pub created_at: DateTime<Utc>,
}

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Eq)]
pub enum AgentMode {
    PlanOnly,
    Supervised,
    Apply,
    Autonomous,
    Evolve,
}
```

8.4 Plan

```
// crates/autoagent-core/src/planning/plan.rs

use crate::editing::file_operation::FileOperation;
use camino::Utf8PathBuf;
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Plan {
    pub objective: String,
    pub summary: String,
    pub files_to_read: Vec<Utf8PathBuf>,
    pub files_to_create: Vec<PlannedFile>,
    pub files_to_modify: Vec<PlannedFile>,
    pub operations: Vec<FileOperation>,
    pub validation_commands: Vec<String>,
    pub risks: Vec<String>,
    pub rollback_strategy: String,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct PlannedFile {
    pub path: Utf8PathBuf,
    pub purpose: String,
}
```

8.5 Validation Report

```
// crates/autoagent-core/src/validation/validation_report.rs

use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct ValidationReport {
    pub passed: bool,
    pub commands: Vec<CommandValidationResult>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct CommandValidationResult {
    pub command: String,
    pub exit_code: Option<i32>,
    pub stdout: String,
    pub stderr: String,
    pub duration_ms: u128,
}
```

9. Workspace Artifacts

```
.agent/
  memory/
    project.json
    decisions.json
    glossary.json
    commands.json
    architecture.json
  plans/
```

```
  <timestamp>-<slug>.plan.json
  <timestamp>-<slug>.plan.md
runs/
  <run-id>/
    run.json
    objective.md
    plan.md
    events.jsonl
    file-operations.json
    validation-report.md
    summary.md
    before/
    after/
patches/
  <run-id>.patch
logs/
  events.jsonl
reports/
  project-analysis.md
tools/
```

9.1 Run Folder Contract

File	Purpose
run.json	Machine-readable metadata for the run, including status, files read, files modified, commands executed, and validation results.
objective.md	Human-readable objective that triggered the run.
plan.md	Human-readable plan used for the run.
events.jsonl	Append-only chronological event log for the run.
file-operations.json	Structured list of file operations applied or proposed.
validation-report.md	Human-readable validation output and command results.
summary.md	Final run summary.
before/	Snapshots of affected files before mutation.
after/	Copies of affected files after mutation.

10. Safety Model

AutoAgent must treat file mutation and command execution as privileged operations. Every read, write, delete, rename, and shell command must go through the policy engine. Built-in tools and future plugins must not bypass the safety layer.

10.1 Path Guard Requirements

- Normalize every path before evaluation.
- Reject paths outside the configured workspace root.
- Reject parent traversal that escapes the workspace.
- Reject blocked paths even if also included by a broad allow rule.
- Resolve symlinks before write operations when possible.
- Never write to .git, target, node_modules, environment files, SSH material, or absolute system paths by default.

10.2 Command Guard Requirements

- Only execute commands explicitly allowed by policy or approved by the user.
- Block risky commands and command fragments by default.
- Record stdout, stderr, exit code, duration, and command string.
- Use the workspace root as the default current working directory.
- Never execute network, remote shell, privilege escalation, destructive, or system ownership commands by default.

11. MVP Implementation Scope

The MVP should focus on the safe mutation engine first. LLM planning can be added after the engine can safely read structured plans, apply changes, validate results, and revert runs.

MVP Capability	Included in 0.1.0
Autoagent.toml loading	Yes
.agent workspace initialization	Yes
doctor command	Yes
file scanner	Yes
policy engine	Yes
snapshot manager	Yes
JSON plan reader	Yes
apply command	Yes
revert command	Yes
JSONL event logs	Yes
LLM-generated planning	No; later milestone
Autonomous repair loop	No; later milestone
Plugin system	No; later milestone
Self-modification mode	No writes by default; plan-only later milestone

12. Recommended Rust Dependencies

```
[dependencies]
anyhow = "1"
thiserror = "1"
clap = { version = "4", features = ["derive"] }
serde = { version = "1", features = ["derive"] }
serde_json = "1"
toml = "0.8"
walkdir = "2"
ignore = "0.4"
globset = "0.4"
similar = "2"
console = "0.15"
dialoguer = "0.11"
indicatif = "0.17"
chrono = { version = "0.4", features = ["serde"] }
uuid = { version = "1", features = ["v4", "serde"] }
sha2 = "0.10"
camino = "1"
```

Optional later dependencies: tokio, reqwest, async-trait, schemars, jsonschema, git2, tree-sitter, tree-sitter-javascript, tree-sitter-typescript, and tree-sitter-rust.

13. Development Roadmap

Version	Milestone	Primary Deliverables
0.1.0	Rust Mutation Engine	Autoagent.toml, .agent workspace, init, doctor, file scanner, policy engine, snapshot manager, plan reader, apply, revert, JSON logs.
0.2.0	Project Analyzer	Language detection, Cargo/package.json detection, dependency summaries, source tree summaries, project report writer.
0.3.0	Planner Interface	Plan command, Markdown plan writer, JSON plan writer, LLM provider interface, prompt builder.
0.4.0	Validation Loop	Run command, apply plan, run validation, inspect failures, repair pass, final report.
0.5.0	Memory	Project memory, decision memory, command memory, architecture memory.
0.6.0	Evolve Mode	Self-modification flag, branch-before-evolve, self-analysis, self-plan, self-apply, self-test.
0.7.0	Plugin System	Rust plugin traits, WASM plugin support, tool registry, plugin manifest.
1.0.0	Stable Release	Stable CLI, stable config schema, stable plan schema, reversible patches, policy enforcement, audit logging.

14. Official Implementation Principles

25. The mutation engine must be safe before the planner becomes powerful.
26. Every file mutation must be structured, policy-validated, snapshotted, logged, and reversible.
27. Every command execution must be explicit, logged, and policy-validated.
28. Self-authoring must remain opt-in and supervised by default.
29. Plans are contracts; the runtime applies validated contracts rather than free-form edits.
30. Rust types should model the domain directly: run states, operations, plans, reports, policies, and errors.
31. The CLI should be user-friendly, but the core should be independently testable.

15. Final Product Statement

AutoAgent is a Rust-native, local-first, policy-driven agent runtime that safely evolves codebases through structured plans, reversible patches, validation commands, project memory, and full audit

logging. Its identity is controlled self-authoring: the ability to understand, modify, validate, and improve software under explicit user rules.

Appendix A: Official Phrases

- A Rust-native local agent runtime for safe, reversible, policy-controlled codebase evolution.
- A recursive developer agent for controlled codebase evolution.
- A self-authoring software agent that can inspect, patch, validate, and evolve codebases under explicit user policy.
- Self-authoring, not uncontrolled self-replicating.
- The safe mutation engine for autonomous software development workflows.

Appendix B: Initial Command Help Draft

```
AutoAgent

USAGE:
  autoagent <COMMAND> [OPTIONS]

COMMANDS:
  init       Initialize AutoAgent in the current workspace
  doctor     Check local system, config, commands, and workspace health
  analyze    Analyze the current project and write a project report
  plan       Create or import a structured implementation plan
  apply      Apply a structured plan through the policy-controlled mutation engine
  run        Plan, apply, validate, and report in a supervised workflow
  evolve     Create a controlled self-authoring plan for AutoAgent itself
  patch      List, show, or save patch artifacts
  revert     Revert a previous AutoAgent run
  memory     Show or manage project memory
  config     Show or validate Autoagent.toml

DEFAULTS:
  - Write operations require approval.
  - Unknown commands require approval.
  - Evolve mode is plan-only unless explicitly applied.
  - Every applied run creates snapshots and logs.
```